BATCH Batch Job Subsystem

User's Guide

BATCH-UG-0102

BATCH Version 1.0.1 *** 29-JUL-2026

Contents

Preface **3**
Intended Audience . . . 3
Document Structure . . . 4
Conventions . . . 5

Chapter 1. Introduction **6**
1.1 Overview . . . 6
1.2 Components . . . 7
1.3 Comparison with Periodic Schedulers . . . 8
1.4 Job States . . . 9
1.5 Job Life Cycle . . . 10

Chapter 2. Installation **11**
2.1 Requirements . . . 11
2.2 Building and Installing . . . 12
2.3 What Is Installed . . . 13
2.4 The Help Library . . . 14
2.5 Starting the Controller with the System . . . 15
2.6 Verifying the Installation . . . 16
2.7 Removal . . . 17

Chapter 3. Configuration **18**
3.1 Qualifiers of the Job Controller . . . 18
3.2 Altering the Service . . . 19
3.3 Settings Fixed at Configuration Time . . . 20
3.4 Where the Utility Looks . . . 21
3.5 What a Job Inherits . . . 22

Chapter 4. Working with Batch Jobs **23**
4.1 Submitting a Batch Job . . . 23
4.2 Passing Data to Batch Jobs . . . 24
4.3 Batch Job Output . . . 25
4.4 Controlling When a Job Runs . . . 26
4.5 Displaying Information About Batch Jobs . . . 27
4.6 Deleting a Batch Job . . . 28

Chapter 5. Command Dictionary **29**
BATCH SUBMIT . . . 30
BATCH SHOW . . . 33
BATCH DELETE . . . 35
BATCH RELEASE . . . 36
BATCH SET ENTRY . . . 37
BATCH PURGE . . . 39
BATCH SYNC . . . 40
BATCH QUEUE . . . 41
BATCH ANALYZE . . . 42

BATCHD 43

Chapter 6. Messages **45**
6.1 Condition Value Format 45
6.2 Message Format 46
6.3 Message Reference 47

Chapter 7. Troubleshooting **55**
7.1 The Controller Cannot Be Reached 56
7.2 The Controller Will Not Start 57
7.3 A Job Does Not Start 58
7.4 A Job Fails 59
7.5 The Queue Is Empty after a Restart 60
7.6 The Log Directory Grows 61
7.7 A Command Is Refused 62
7.8 The Controller Is Slow to Answer 63
7.9 Collecting Information for a Report 64

Appendix A. Files **65**
Environment Variables 66

Appendix B. Time Specification Formats **67**
B.1 Delta Time 68
B.2 Absolute Time 69
B.3 Keywords 70

Preface

Intended Audience

This manual is intended for the user who submits shell scripts for unattended execution and for the system manager who installs and operates the job controller. Familiarity with a POSIX shell is assumed; nothing else is. No prior knowledge of OpenVMS is required, although the reader acquainted with the OpenVMS `SUBMIT` and `SHOW QUEUE` commands will recognize both the terminology and the command syntax.

The programmer who is to alter the subsystem wants the *Programmer's Guide* instead. This manual says nothing about how the software is built.

Document Structure

This manual contains the following chapters and appendixes:

Part	Contents
Chapter 1	Introduces the subsystem, its components, and the job life cycle
Chapter 2	Installing the subsystem and starting the controller with the system
Chapter 3	Configuring the controller and what a job inherits from it
Chapter 4	Describes how to perform the common batch job operations
Chapter 5	Contains the command dictionary, one entry per command
Chapter 6	Describes the format of a message and lists every message with its meaning and the action it calls for
Chapter 7	Troubleshooting: a symptom, what to look at, and what to do
Appendix A	Lists the files created and used at run time
Appendix B	Describes the accepted time specification formats

The reader installing the subsystem for the first time wants Chapter 2, then Chapter 3. The reader who has it running and wants to use it wants Chapter 4. The reader holding a message wants Chapter 6, and the one holding a symptom, Chapter 7.

Conventions

The following conventions are used in this manual:

Convention	Meaning
`$`	The shell prompt. Text following the prompt is entered by the user.
italics	A term being defined, or a variable to be replaced by a value.
`UPPERCASE`	A command verb, qualifier, or condition value mnemonic.
`[ ]`	Optional syntax elements. Do not type the brackets.
`{ }`	A required choice among the enclosed items. Do not type the braces.
`, . . .`	The preceding element may be repeated.

Commands and qualifiers are shown in uppercase for readability but may be entered in any case.

Note

Sample output in this manual is reproduced exactly as the software produces it.

Note

The subsystem keeps time in **UTC** and shows it by the clock of whoever is reading. Every time displayed by the `SHOW` command, and every time supplied to the `/AFTER` qualifier, is in the local time zone; the queue journal holds UTC, which is what makes a queue survive a change of zone of the node. Time conversion is performed by the STARLET services `UTIL$BINTIM`, `UTIL$ASCTIM`, `UTIL$GETTIM` and `UTIL$ADD_TIMES`.

Chapter 1. Introduction

1.1 Overview

The BATCH subsystem executes shell scripts as *batch jobs*. A batch job is a script that has been placed in a queue and is executed later, without a terminal, under the control of a permanently resident process called the *job controller*.

Submitting a script does not execute it. Submission places an entry in a queue and returns an *entry number* that identifies the job for the remainder of its life. The job controller decides when the job actually runs, based on the number of jobs already executing, the priority assigned to the job, and the time at which the job is permitted to start.

1.2 Components

The subsystem consists of two images:

Image	Function
`batch`	The user interface. Parses commands, sends requests to the job controller, and displays results.
`batchd`	The job controller. Owns the queue, schedules jobs, creates processes, and records job completion.

The two images communicate over a UNIX domain socket in the subsystem root directory. The job controller identifies the sender of every request from the credentials supplied by the kernel; it does not trust identification carried in the request itself.

1.3 Comparison with Periodic Schedulers

Users familiar with `cron` should note the following differences:

- A job is identified by an entry number assigned at submission time, not by a process identifier. The entry number exists before any process does.
- A queue has *states*. A job may wait for an execution slot or for a designated start time before it runs.
- The number of jobs executing at one time is limited. Submitting more jobs than the limit allows does not overload the system; the excess jobs wait.
- A job that terminates abnormally *remains in the queue* together with its log file, and is displayed by the `SHOW` command without any qualifier.

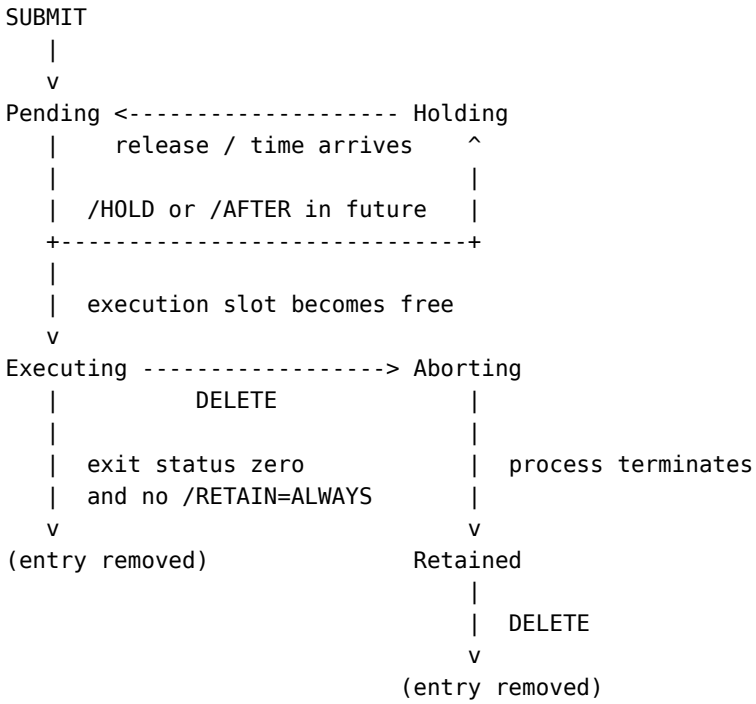
1.4 Job States

A job occupies exactly one of the following states:

State	Meaning
`Pending`	The job is eligible to run and is waiting for an execution slot.
`Holding`	The job is withheld, either by the `/HOLD` qualifier or because its `/AFTER` time has not arrived.
`Executing`	A process has been created for the job and is running.
`Aborting`	The job has been sent a termination signal and has not yet terminated.
`Retained`	The job has terminated and its queue entry has been kept for inspection.

1.5 Job Life Cycle

The permitted transitions are shown below.



A job that completes successfully leaves the queue, as it does in OpenVMS. To keep the entry of a successful job, submit it with `/RETAIN=ALWAYS`.

Chapter 2. Installation

2.1 Requirements

Operating system	Linux. The subsystem uses UNIX domain sockets, the `SO_PEERCRED` socket option, `daemon(3)` and `poll(2)`; a build elsewhere is stopped during configuration.
Build tools	CMake 3.16 or later and a C compiler supporting gnu11.
Service supervisor	systemd, if the controller is to start with the system. It is not otherwise required.
Privilege	The installation writes to system directories and needs the privileged account. Building and testing do not.

Two packages must be installed beforehand: **StarLet**, which is UTILITY$ROUTINES, and **HELP**. Neither is carried with these sources and neither is optional - without the first nothing compiles, and without the second the `BATCH HELP` command does not.

2.2 Building and Installing

```
$ cmake -S . -B build
$ cmake --build build -j
$ ctest --test-dir build --output-on-failure
# cmake --install build
```

The tests are worth running before installing. They bring up a controller of their own on a temporary directory, exercise the queue and take it down again; they touch neither the working configuration nor any account.

To install elsewhere than `/usr/local`, state it at configuration time:

```
$ cmake -S . -B build -DCMAKE_INSTALL_PREFIX=/opt/batch
```

Section 3.3 lists the other settings that are fixed at configuration time.

2.3 What Is Installed

Path	Contents
prefix`/bin/batch`	the command line utility
prefix`/sbin/batchd`	the job controller
prefix`/share/doc/BATCH/`	the manuals and the summary, as markdown and, when asked for, as PDF
`/lib/systemd/system/batchd.service`	the unit of the controller
`/var/spool/batch/`	the subsystem root: the queue journal and the socket
`/var/spool/batch/log/`	the log files of jobs

The location of the unit is `BATCH_K_SYSTEMDDIR`; setting `BATCH_M_SYSTEMD` to `OFF` leaves it out altogether, which is what a system without systemd calls for.

2.4 The Help Library

The installation puts the help library of the subsystem in `share/help` and rebuilds the catalogue that the HELP utility shares between its users:

```
-- %BATCH-I-SYSCACHE, the help catalogue has been rebuilt
```

That catalogue is harvested from the libraries of the directory once, so a library installed after it was built is not seen until it is built again. Two things follow.

A **staged installation** — one made with `DESTDIR` set, as a package is assembled — does not build it, and says so: a catalogue built from a staging directory would describe that directory rather than the system. Whoever installs the package rebuilds it, ordinarily from the post-install script:

> The image of the HELP browser is named `vhelp`, and its librarian `vhelptran`: `help` is a builtin of the Bourne-again shell and of its kin, which look a builtin up before the search path, so an image of that name could not be reached by ordering the path. The facility ships a shell snippet that turns the builtin off for those who would rather type `help`.

```
# vhelp /SYSCACHE
```

A **failure to rebuild** it is reported and passed over. It costs nothing but this: `help submit` asked of the general utility does not find the topic until the catalogue is built. The `BATCH HELP` command is unaffected either way — it reads the library directly and consults no catalogue.

2.5 Starting the Controller with the System

The unit is installed but **not enabled**. Whether the controller starts with the system is for whoever administers it to decide.

```
# systemctl enable --now batchd
# systemctl status batchd
```

The controller runs in the foreground under the supervisor, so its diagnostics reach the journal of the system rather than a log of its own:

```
# journalctl -u batchd -f
```

Note

> The unit carries `KillMode=process`, so stopping the service signals the controller alone. A job is a session of its own and is meant to outlive the controller; the default of killing the whole control group would take every running job with it.

Without systemd the controller is started as any other daemon, and detaches by itself:

```
# /usr/local/sbin/batchd /ROOT=/var/spool/batch /JOB_LIMIT=8
```

2.6 Verifying the Installation

```
$ printf 'echo it works\n' > /tmp/probe.sh
$ batch submit /tmp/probe.sh /NAME=probe /KEEP
Job probe (queue SYS$BATCH, entry 1) queued

$ batch show /all
There are no jobs in the queue

$ cat /var/spool/batch/log/probe_1.log
it works
```

The queue is empty because the job succeeded and its entry left the queue, which is the behaviour described in Section 1.5. The log survives because `/KEEP` was given.

Should the first command answer `%BATCH-E-NOCONNECT`, the controller is not running; Section 7.1 takes that up.

2.7 Removal

```
# cmake --build build --target uninstall
```

What was installed is read from the manifest written at installation time, so nothing is guessed at and nothing that was not installed is touched. The subsystem root and the log directory are removed only while they are empty: a removal of the software is not a reason to discard the queue journal or the logs of jobs.

Disabling the service is left to the administrator and is stated at the end of the removal:

```
# systemctl disable --now batchd
```

Chapter 3. Configuration

3.1 Qualifiers of the Job Controller

The controller takes its settings from its command line. Section 3.2 explains where to put them when it runs as a service.

Qualifier	Default	Meaning
`/ROOT=`*directory*	`/var/spool/batch`	The subsystem root. The queue journal, the socket and the log directory live there.
`/JOB_LIMIT=`*n*	16	Jobs that may execute at one time. Independent of the number of processors.
`/LCORE=`*list*	—	Confines the batch load to the processors named and spreads it over them, one to each job in turn. `(0,3,4)` and `0,3,4` are both taken. A job naming `/CPU` keeps what it named.
`/QUEUE=`*name*	`SYS$BATCH`	Name of the default queue.
`/LOG_RETAIN=`*days*	7	How long a job log left behind is kept. Zero stops the sweep.
`/FOREGROUND`	—	Do not detach. Intended for a service supervisor and for testing.
`/VERBOSE`	—	Issue messages of informational severity as well.

A qualifier may be introduced by a slash or a hyphen and abbreviated to any unambiguous leading portion; the initial letters are all distinct, so `-r=`, `-j=`, `-q=`, `-l=`, `-f` and `-v` serve. A value is attached to its qualifier by an equals sign and is never separated from it by a blank.

3.2 Altering the Service

Do not edit the installed unit: the next installation overwrites it. Use a drop-in.

```
# systemctl edit batchd
```

```
[Service]
ExecStart=
ExecStart=/usr/local/sbin/batchd /FOREGROUND /ROOT=/var/spool/batch /JOB_LIMIT=8 /LOG_RETAIN=30
```

The empty `ExecStart=` is required: without it the second line adds a command rather than replacing the one the unit carries.

```
# systemctl daemon-reload
# systemctl restart batchd
```

Restarting the controller does not lose the queue — it is read back from the journal — but a job that was executing is orphaned unless it was submitted with `/RESTART`. The SUBMIT entry in Chapter 5 takes that up.

3.3 Settings Fixed at Configuration Time

These are compiled in and are altered by configuring the build afresh:

Variable	Default	Meaning
`BATCH_K_SPOOLROOT`	`/var/spool/batch`	The root the images assume when none is named
`BATCH_K_SHELL`	`/bin/sh`	The command interpreter created for a job
`BATCH_K_FACILITY`	`BATCH`	The facility name appearing in messages
`BATCH_K_SYSTEMDDIR`	`/lib/systemd/system`	Where the unit is installed
`BATCH_M_SYSTEMD`	`ON`	Whether the unit is installed at all
`BATCH_M_PDF`	`OFF`	Render the manuals as PDF and install them as well
`BATCH_M_TESTS`	`ON`	Whether the tests are built
`BATCH_M_STRICT`	`OFF`	Treat compiler diagnostics as errors

```
$ cmake -S . -B build -DBATCH\$K_SPOOLROOT=/opt/batch -DBATCH\$M_SYSTEMD=OFF
```

The backslash keeps the shell from expanding the dollar sign; CMake itself permits it in a name.

3.4 Where the Utility Looks

The utility takes the subsystem root from the environment variable `BATCH_ROOT`, and from the compiled-in default when that is not set. A controller started with `/ROOT` other than the default therefore needs its users to set the variable to match:

```
$ export BATCH_ROOT=/opt/batch
```

A mismatch shows itself as `%BATCH-E-NOCONNECT`, since the utility looks for the socket in a directory where none was created.

3.5 What a Job Inherits

A job is created by the controller and takes from it what it does not name itself:

Processor affinity	Inherited. `/CPU` overrides it for an individual job.
Nice value	Inherited. Cannot be set per job.
Resource limits	Inherited. Cannot be set per job.
Control group	Inherited. Cannot be set per job.
Identity	**Not** inherited: the job runs as whoever submitted it.
Working directory	**Not** inherited: the job starts where it was submitted from.
Environment	**Not** inherited: the job is given `BATCH_*`, `P1` … `P8` and nothing else of the controller's.

The controller states the affinity mask it runs under when it starts, so a controller started under `taskset`, or with `CPUAffinity=` in its unit, does not constrain its jobs unnoticed:

```
%BATCH-I-INFO, the controller runs on processors 0-3, and a job inherits that
```

Note that a control group limit such as `CPUQuota=` applies to the controller and every job it creates, taken together.

Chapter 4. Working with Batch Jobs

4.1 Submitting a Batch Job

To submit a script, use the `SUBMIT` command:

```
$ batch submit /opt/jobs/nightly.sh
Job nightly (queue SYS$BATCH, entry 17) queued
```

The command displays the name assigned to the job, the queue it was placed in, and the entry number. Record the entry number; it is required by the `DELETE` and `RELEASE` commands.

By default the job name is the file name of the script with any `.sh` suffix removed. Use the `/NAME` qualifier to assign a different name:

```
$ batch submit /opt/jobs/nightly.sh /NAME=reindex
Job reindex (queue SYS$BATCH, entry 18) queued
```

The script specification is resolved to an absolute path at submission time. Note also that the job starts in the *working directory that was current when the job was submitted*, not in the directory current when the job begins execution. A script that relies on relative paths will therefore behave as it would if run interactively from the same directory.

4.2 Passing Data to Batch Jobs

Use the `/PARAMETERS` qualifier to pass up to eight values to a job:

```
$ batch submit /opt/jobs/backup.sh '/PARAMETERS=(full,/srv/data,verbose)'
```

Each value is made available to the script in two ways: as the environment variables `P1` through `P8`, and as the positional arguments `$1` through `$8`.

```
#!/bin/sh
echo "mode  = ${P1}"          # full
echo "target= $2"             # /srv/data
```

The values are passed to the shell as separate elements of the argument vector. They are never concatenated into a command line, so a value containing shell metacharacters cannot cause unintended command execution.

Note

> The parentheses in `/PARAMETERS=(...)` are interpreted by the shell that invokes `batch`. Enclose the whole qualifier in single quotation marks, as shown above, or omit the parentheses and write `/PARAMETERS=full,/srv/data,verbose`.

Three further environment variables are supplied to every job:

Variable	Contents
`BATCH_ENTRY`	The entry number of the job
`BATCH_NAME`	The job name
`BATCH_QUEUE`	The name of the queue
`BATCH_RESTART`	`TRUE` when the job is run again after a restart of the controller
`BATCH_RESTARTS`	Number of times the job has been restarted

4.3 Batch Job Output

A batch job has no terminal. Its standard input is connected to the null device, and its standard output and standard error are directed to a *log file*.

Unless the `/LOG_FILE` qualifier is specified, the log file is created in the `log` subdirectory of the subsystem root and is named for the job and its entry number:

```
/var/spool/batch/log/nightly_17.log
```

The disposition of the log file depends on how the job terminates:

Outcome	Queue entry	Log file
Exit status zero	Removed	Deleted, unless `/KEEP` was specified
Nonzero exit status	Retained	Kept
Terminated by signal	Retained	Kept
Any outcome, `/RETAIN=ALWAYS`	Retained	Kept

This is the rule that gives the `SHOW` command its default behavior: what remains in the queue is precisely the set of jobs that are still running or that failed.

To retain the log of a successful job for inspection, use `/KEEP`:

```
$ batch submit /opt/jobs/nightly.sh /KEEP
```

4.4 Controlling When a Job Runs

2.4.1 Deferring Execution

The `/AFTER` qualifier prevents a job from starting before a designated time. The job is placed in the `Holding` state and is released automatically when the time arrives:

```
$ batch submit /opt/jobs/nightly.sh '/AFTER=+1-02:00'
$ batch submit /opt/jobs/report.sh  /AFTER=29-JUL-2026:03:00
$ batch submit /opt/jobs/cleanup.sh /AFTER=TOMORROW
```

Appendix B describes the accepted time formats in full.

2.4.2 Holding and Releasing a Job

The `/HOLD` qualifier places a job in the queue but prevents it from being considered for execution until it is released explicitly:

```
$ batch submit /opt/jobs/migrate.sh /NAME=migrate /HOLD
Job migrate (queue SYS$BATCH, entry 19) queued

$ batch release 19
%BATCH-S-ENTRELEASED, entry 19 released from the holding state
```

Releasing a job also clears any `/AFTER` time associated with it, so a released job becomes eligible immediately.

2.4.3 Priority

The `/PRIORITY` qualifier controls the order in which pending jobs are selected. The job controller selects the pending job with the highest priority; among jobs of equal priority it selects the one with the lowest entry number, so that jobs of equal priority are served in the order they were submitted.

```
$ batch submit /opt/jobs/urgent.sh /PRIORITY=200
```

Note

> `/PRIORITY` is the priority of the job *within the queue*. It does not affect the scheduling priority of the process created for the job, and has no effect once the job is executing. Use `/CPU` to bind a job to particular processors; the nice value, the resource limits and the control group are inherited from the job controller and cannot be set per job.

2.4.4 Altering a Job in the Queue

What `SUBMIT` settled need not be settled for good. The `SET ENTRY` command alters the attributes of a job that is in the queue, in place: the entry keeps its number and its place in the order, and only the attributes named on the command line are touched.

```
$ batch set entry 19 /PRIORITY=200
%BATCH-S-ENTMODIFIED, entry 19 modified: priority=200

$ batch set entry 19 /AFTER=TOMORROW /NOHOLD
%BATCH-S-ENTMODIFIED, entry 19 modified: after= 9-SEP-2026 00:00:00, nohold
```

A job that waits may have anything altered. A job that executes may have only what concerns its disposition afterwards altered — `/RETAIN`, `/KEEP`, `/RESTART` — and a retained job nothing. See the command dictionary for the rules in full.

4.5 Displaying Information About Batch Jobs

The `SHOW` command displays the queue. Without qualifiers it displays the jobs that are executing, being aborted, or retained after failure:

```
$ batch show

 Entry  Job name            State      Submitted            Elapsed   Status
 -----  ------------------- ---------- -------------------- --------- ----------
    17  nightly             Executing  28-JUL-2026 10:01:05 00:04:12  -
    18  reindex             Retained   28-JUL-2026 10:01:05 00:00:07  exit=7

Total entries: 2
```

The `Status` column shows `exit=`*n* for a job that returned a nonzero exit status, `signal=`*n* for a job terminated by a signal, and a hyphen otherwise.

Specify `/ALL` to include jobs that are waiting:

```
$ batch show /all
```

Specify `/FULL` to obtain a complete description of each job instead of one line per job:

```
$ batch show /full

Entry 17, job nightly, queue SYS$BATCH
    State ............ Executing, pid 4412
    Owner ............ smith (uid 1000, gid 1000)
    Priority ......... 100
    Script ........... /opt/jobs/nightly.sh
    Working directory  /home/smith
    Log file ......... /var/spool/batch/log/nightly_17.log
    Submitted ........ 28-JUL-2026 10:01:05
    Started .......... 28-JUL-2026 10:01:05
    Parameters ....... P1 = "full"
                       P2 = "/srv/data"
```

By default you see only your own jobs. Specify `/EVERYONE` to display the jobs of all users.

4.6 Deleting a Batch Job

The `DELETE` command removes a queue entry. Its effect depends on the state of the job:

State	Effect
`Pending`, `Holding`, `Retained`	The entry is removed and the log file deleted unless `/KEEP` was specified
`Executing`	A termination signal is sent to the job; the entry is disposed of when the job terminates

```
$ batch delete 17
Entry 17: operation completed
```

The signal is sent to the *process group* of the job, not to a single process. The job is the leader of its own process group, so every process it created is terminated with it.

You may delete only your own jobs, unless you are the superuser.

Chapter 5. Command Dictionary

This chapter describes each command in detail. The commands are:

```
BATCH SUBMIT
BATCH SHOW
BATCH DELETE
BATCH RELEASE
BATCH SET ENTRY
BATCHD
```

The command line is parsed by the CLI services of the UTILITY$ROUTINES subproject. A qualifier may be introduced by either a slash or a hyphen and may be abbreviated to any unambiguous leading portion; `/PARAM=` and `/PARAMETERS=` are equivalent, and an ambiguous abbreviation is rejected naming the candidates. `/RESTART` and `/RETAIN` share their first two letters, so an abbreviation of either takes three. A value is attached to its qualifier by an equals sign.

The parameter of a command and its qualifiers may appear in any order.

Values are validated against the type declared for the qualifier: `/PRIORITY` and `/ENTRY` accept a number only, and `/RETAIN` one of its keywords only.

The `batch` image returns 0 to the shell if the operation succeeded and 1 if it did not.

BATCH SUBMIT

Queues a shell script for execution as a batch job.

Format

```
batch submit  script-specification
```

Parameter

script-specification

Specifies the shell script to be executed. The specification is resolved to an absolute path when the command is entered. The file must exist and be readable both by the user entering the command and by the job controller.

Because a file specification and a qualifier both begin with a slash, the command distinguishes them by name: an argument beginning with a slash is treated as a qualifier only if the text up to the first equals sign or colon names a known qualifier. Therefore `/tmp/job.sh` is recognized as a file specification and `/NAME=x` as a qualifier.

Description

The SUBMIT command places an entry in a batch queue and displays the entry number assigned to it. It does not wait for the job to execute.

The following attributes are recorded at submission time and cannot be changed afterwards:

- the absolute path of the script;
- the working directory current when the command was entered;
- the identity of the submitting user, which is the identity under which the job will run.

If the queue table is full, or the script is not accessible, the command fails and no entry is created.

Qualifiers

/AFTER=*time*

Specifies the earliest time at which the job may begin execution. The job is placed in the `Holding` state until the time arrives, whereupon the job controller releases it without further intervention. See Appendix B for the accepted formats.

/CPU=*processor-list*

Binds the job to the processors named, in the notation `taskset(1)` uses: a comma-separated series of numbers and of ranges, `0-3,5,8-11`. Without the qualifier the job inherits the affinity mask of the job controller, which the controller states in its log when it starts.

The list is verified when the command is entered, so a mistake in it is reported at the terminal rather than turning into a job that fails.

/HOLD

Places the job in the `Holding` state. The job is not considered for execution until it is released by the `RELEASE` command. Use this qualifier to submit a job whose start time is not yet known.

/KEEP

Retains the log file after the job completes successfully. By default the log file of a successful job is deleted together with its queue entry.

/LOG_FILE=*file-specification*

Specifies the file to which the standard output and standard error of the job are directed. The specification is resolved to an absolute path when the command is entered; the file itself need not exist, but its directory must. By default the log file is created in the `log` subdirectory of the subsystem root and is named *job-name*`_`*entry*`.log`.

/NAME=*job-name*

Specifies the name of the job, up to 63 characters. By default the job name is the file name of the script with any `.sh` suffix removed. The job name appears in displays and forms part of the default log file name.

/PARAMETERS=(*value*[,...])

Specifies up to eight values to be passed to the job. Each value may be up to 255 characters. The values are supplied to the script both as the environment variables `P1` through `P8` and as the positional arguments `$1` through `$8`. If more than eight values are given, the excess values are discarded and a warning is issued; the job is still queued.

/PRIORITY=*n*

Specifies the scheduling priority of the job within the queue, in the range 1 to 255. The default is 100. The job controller selects the pending job of highest priority; jobs of equal priority are selected in order of entry number. This qualifier does not affect the priority of the process created for the job.

/QUEUE=*queue-name*

Specifies the queue in which the job is to be placed. The default is the queue name with which the job controller was started, normally `SYS$BATCH`.

/RESTART

Specifies that the job is to be run again if the job controller is stopped while the job is executing. Such a job goes back into the queue when the controller is started next; without the qualifier it is kept in the `Retained` state, marked as orphaned, and is not run again.

The process left behind by the earlier instance of the controller is neither sought out nor stopped: it belongs to a session of its own and may well have finished by then. What the qualifier promises is that the job will be run, not that it will be run once, so a restartable job should be written such that running it twice does no harm.

The job is told of a restart through the environment variables `BATCH_RESTART`, which reads `TRUE` or `FALSE`, and `BATCH_RESTARTS`, which counts them. This answers to the `$RESTART` symbol of OpenVMS.

/USER=*name*

Runs the job as the user named rather than as the submitter. Only the privileged user may ask for it: the identity is settled by the job controller from what the kernel reports about the sender of the request, not from anything the request says.

Every use of the qualifier and every refusal is recorded in the log of the controller, naming who asked and for whom.

The job starts in the home directory of that user rather than in the directory it was submitted from, and its log goes to the log directory of the subsystem.

/RETAIN={ALWAYS | ERROR | NEVER}

Specifies the condition under which the queue entry is kept after the job terminates. The keyword may be abbreviated to three characters.

Keyword	Effect

`ALWAYS`	The entry is always kept
`ERROR`	The entry is kept only if the job failed. This is the default.
`NEVER`	The entry is never kept, even if the job failed

Examples

1.

```
$ batch submit /opt/jobs/nightly.sh
Job nightly (queue SYS$BATCH, entry 17) queued
```

The `SUBMIT` command in this example queues the script `nightly.sh` for execution. The job takes its name from the script, is placed in the default queue, and is eligible for execution immediately. The entry number 17 identifies the job in subsequent commands.

2.

```
$ batch submit /opt/jobs/backup.sh /NAME=weekly '/PARAMETERS=(full,/srv/data)' \
        '/AFTER=+1-02:00' /PRIORITY=150 /KEEP
Job weekly (queue SYS$BATCH, entry 18) queued
```

This command queues a job named `weekly` that may not begin before 26 hours have elapsed. Two parameters are passed to the script. The job is given a priority above the default, so that when its time arrives it is selected in preference to jobs of default priority. The log file is kept even if the job succeeds.

3.

```
$ batch submit /opt/jobs/migrate.sh /HOLD
Job migrate (queue SYS$BATCH, entry 19) queued
$ batch show /all
...
    19  migrate              Holding    28-JUL-2026 11:20:03 00:00:00  -
```

The job is queued but withheld from execution. Because it is not executing and has not failed, it is displayed only when `/ALL` is specified.

BATCH SHOW

Displays the contents of a batch queue.

Format

```
batch show
```

Parameters

None.

Description

The `SHOW` command displays queue entries. Which entries are displayed is governed by the `/ALL` and `/EVERYONE` qualifiers; how much is displayed for each is governed by `/FULL`.

Without `/ALL`, the command displays jobs in the `Executing`, `Aborting`, and `Retained` states, that is, the jobs that are running and the jobs that failed. Because the entry of a successful job is removed from the queue, such a job is not displayed even when `/ALL` is specified. Submit the job with `/RETAIN=ALWAYS` if its entry is to survive successful completion.

In the brief display, the `Elapsed` column shows the time from the start of the job to its termination, or to the present moment if the job is still executing.

Qualifiers

/ALL

Includes jobs in the `Pending` and `Holding` states in the display.

/EVERYONE

Displays the jobs of all users. By default only the jobs of the user entering the command are displayed.

/FULL

Displays a complete description of each job, including its owner, priority, script specification, working directory, log file specification, timestamps, exit status, and parameters.

/QUEUE=*queue-name*

Restricts the display to jobs in the specified queue.

Examples

1.

```
$ batch show

 Entry  Job name             State      Submitted            Elapsed   Status
 -----  -------------------- ---------- -------------------- --------- ----------
    17  nightly              Executing  28-JUL-2026 10:01:05 00:04:12  -
    18  reindex              Retained   28-JUL-2026 10:01:05 00:00:07  exit=7

Total entries: 2
```

One job is running; one failed with an exit status of 7 and has been kept in the queue together with its log file.

2.

```
$ batch show /all /everyone /queue=SYS$BATCH
```

Displays every entry in the queue `SYS$BATCH`, in any state, for every user.

BATCH DELETE

Deletes a queue entry, terminating the job if it is executing.

Format

```
batch delete  entry-list  [/CONFIRM]
```

Parameter

entry-list

Specifies the entries, as displayed by the `SUBMIT` and `SHOW` commands. A list is written as DCL writes one: numbers and ranges separated by commas, the parentheses optional — `1,3,4` and
`(1,3-5)` alike. Every element is applied whatever became of those before it, and the command exits with a failure status if any of them failed.

Qualifier

/CONFIRM

Asks before it goes ahead. Anything but a word beginning with Y is a refusal: what DELETE does cannot be undone, and a list makes a slip cost more than one entry. Redirected input is taken as confirmed — a question put into a file would wait for a key nobody is at hand to press — and `/NOCONFIRM` says the same explicitly.
The number may also be supplied with the `/ENTRY=` qualifier.

Description

If the job is executing, a termination signal is sent to its process group and the job enters the `Aborting` state; the entry is disposed of when the process terminates, and the log file is kept. Otherwise the entry is removed at once and its log file deleted unless `/KEEP` was specified when the job was submitted.

You may delete only your own jobs. The superuser may delete any job.

Example

```
$ batch delete 17
Entry 17: operation completed
```

BATCH RELEASE

Releases a job from the `Holding` state.

Format

```
batch release  entry-list
```

Parameter

entry-number

`Specifies the entry number of the job.`

Description

The job is placed in the `Pending` state and becomes eligible for execution. Any `/AFTER` time associated with the job is cleared, so a released job is not withheld a second time.

The command fails if the job is not in the `Holding` state.

Example

```
$ batch release 19
Entry 19: operation completed
```

BATCH SET ENTRY

Alters the attributes of a job that is in the queue, in place.

Format

```
batch set entry  entry-list  [/PRIORITY=n] [/AFTER=time | /NOAFTER] [/HOLD | /NOHOLD]
                             [/NAME=name] [/RETAIN=keyword] [/KEEP | /NOKEEP]
                             [/RESTART | /NORESTART] [/CPU=list]
```

Parameter

entry-list

`The entries to alter, written as for DELETE: numbers and ranges separated by commas.`

Description

The entry keeps its number and its place in the order; nothing is deleted and nothing is submitted anew. Only the attributes a qualifier names are touched, and the confirmation lists them so that it can be checked against what was typed. A command that names none is refused.

What may be altered depends on the state of the job:

State	May be altered
`Pending`, `Holding`	everything
`Executing`, `Aborting`	`/RETAIN`, `/KEEP`, `/RESTART` only
`Retained`	nothing

A request that names an attribute the state forbids is refused whole, so that an entry is never half altered.

`/HOLD` and `/AFTER` work together as they do at submission. `/HOLD` withholds a pending job; `/NOHOLD` lets a held one go and clears its time, as `RELEASE` does. `/AFTER` with a time in the future withholds a pending job until then. A held job that carries a time is let go when the time comes or when `/NOAFTER` clears it; one held by `/HOLD` alone waits for `RELEASE` or `/NOHOLD`. `/HOLD` and `/NOAFTER` together therefore mean held, without a time. Where both are given, holding is applied first and the time second, so `/NOHOLD /AFTER=t` lets the job go and withholds it until t.

Qualifiers

/PRIORITY=n — the priority within the queue, 1 to 255.

/AFTER=time, **/NOAFTER** — the time before which the job is not to start, in the formats of Appendix B; `/NOAFTER` clears it.

/HOLD, **/NOHOLD** — withholds the job, or lets a held one go.

/NAME=name — the name of the job, up to 63 characters. It changes what the displays show and nothing else: the log file was named when the job was submitted and keeps its name.

/RETAIN=keyword — `ALWAYS`, `ERROR` or `NEVER`, as for SUBMIT.

/KEEP, **/NOKEEP** — whether the log of a successful job is kept.

/RESTART, **/NORESTART** — whether the job is run again after a restart of the controller.

/CPU=list — the processors the job is bound to, in the notation of SUBMIT.

Examples

```
$ batch set entry 17 /PRIORITY=200
%BATCH-S-ENTMODIFIED, entry 17 modified: priority=200

$ batch set entry 17 /HOLD /NOAFTER
%BATCH-S-ENTMODIFIED, entry 17 modified: noafter, hold

$ batch set entry 17 /NAME=x
%BATCH-E-SUBREJECT, the controller rejected the request: NOTMODIFIABLE (08010362)
```

The last is a job already executing: its name describes a start that has happened.

BATCH PURGE

Removes the entries left behind by jobs that have finished.

```
BATCH PURGE [/QUEUE=queue] [/CONFIRM]
```

A queue that keeps its failures needs tidying: an entry retained after a job failed stays until somebody has looked at it — and the looking is done, but the entries remain.

Only a job that has finished is taken. A job still executing is not touched: PURGE tidies what is over, it does not stop what is not. Your own entries, unless you are the privileged user, who may dispose of any.

```
$ batch purge
%BATCH-S-PURGED, 3 entries removed from the queue
```

Qualifier	Effect
`/QUEUE=`*queue*	Restricts the purge to the queue named
`/CONFIRM`	Asks before it goes ahead

BATCH SYNC

Waits for a job to end and reports how it ended.

```
BATCH SYNC entry-number [/TIMEOUT=seconds]
```

Meant for a script that has work to do after a job and no other way of knowing when the job is done. The exit status carries the answer, so the caller need not read the display at all: zero where the job succeeded, non-zero where it failed, where the wait ran out, or where there is no such entry.

```
batch submit nightly.sh /NAME=nightly
batch sync 17 /TIMEOUT=3600 || echo "the nightly job did not go well"
```

An entry that has left the queue is taken to have succeeded, that being what makes an entry leave. Without `/TIMEOUT` the wait has no limit.

BATCH QUEUE

Pauses a queue, or lets it go on again. Only the privileged user may ask for it: pausing withholds the work of everybody who uses the queue.

```
BATCH QUEUE queue-name {/PAUSE | /RESUME}
```

A paused queue starts nothing. Its entries are not disturbed and its executing jobs are left to finish: pausing is what one does to a queue whose jobs must wait without their entries being touched.

BATCH ANALYZE

Examines the queue journal and reports what is wrong with it.

```
BATCH ANALYZE [journal-specification] [/DUMP]
```

The file is read directly rather than through the job controller, so the command serves where the controller will not start — which is when it is wanted most — and serves equally on a copy of a journal taken from another machine. Nothing is written.

`/DUMP` displays each record as well: its entry, name, size, state, owner, priority, queue, the time it was queued and the script it runs, how a job that has ended ended, and what a held one waits for.

BATCHD

Starts the job controller.

Format

```
batchd  [/ROOT=directory] [/JOB_LIMIT=n] [/QUEUE=queue-name] [/FOREGROUND] [/VERBOSE]
        [/LOG_RETAIN=days]
```

Description

The job controller owns the queue. It must be running for any `batch` command to succeed.

On startup the controller creates its root directory and log subdirectory if necessary, restores the queue from the journal, and creates the socket on which it accepts requests. Jobs that were executing when a previous instance stopped are marked as orphaned and placed in the `Retained` state, because their processes are not children of the new instance and their exit status can no longer be obtained. A job submitted with `/RESTART` is instead put back in the queue and run again.

Unless `/FOREGROUND` is specified, the controller detaches from the terminal and directs its diagnostics to `batchd.log` in the subsystem root directory. That log is rewound once it exceeds a megabyte; the log of an individual job is not, and is removed only with the queue entry it belongs to.

Qualifiers

The command line is parsed by the CLI services of the UTILITY$ROUTINES subproject. A qualifier may be introduced by either a slash or a hyphen and may be abbreviated to any unambiguous leading portion of its name; the initial letters are all distinct, so a single letter suffices. A value is attached to its qualifier by an equals sign, never separated from it by a blank.

/FOREGROUND

Runs in the foreground and leaves the standard streams alone. Intended for testing and for operation under a service supervisor. Otherwise the controller detaches and directs its diagnostics to `batchd.log` in the subsystem root directory.

/JOB_LIMIT=*n*

Specifies the number of jobs that may execute simultaneously. The default is 4, independent of the number of processors. Values outside the range 1 to 512 are replaced by the default. A value that is not a number is rejected during parsing.

/LOG_RETAIN=*days*

Specifies how long a job log left behind in the log directory is kept. The controller sweeps that directory when it starts and once an hour thereafter, removing files older than the period that belong to no entry now in the queue. The default is 7 days; zero disables the sweep.

A log named by `/LOG_FILE` outside the log directory of the subsystem is never touched.

/QUEUE=*queue-name*

Specifies the name of the default queue. The default is `SYS$BATCH`.

/ROOT=*directory*

Specifies the subsystem root directory, overriding the value established when the software was built.

/VERBOSE

Issues messages of informational severity.

/QUEUES

Displays the queues rather than the jobs: each with its limit, what it has executing, pending and holding, and whether it is paused.

```
 Queue                Limit  Exec  Pend  Hold  State
 -------------------- ----- ----- ----- ----- --------
 SYS$BATCH               16     0     0     0  open
 slow                     1     1     1     0  paused
```

/MANAGER

Displays the state of the job controller rather than the contents of the queue: the image it runs, its process and how long it has been up, the root it serves, the job limit and what is executing against it, the counts by state, the next entry number, and the pool of processors it was given.

Examples

```
# batchd /ROOT=/var/spool/batch /JOB_LIMIT=8
# batchd -r=/var/spool/batch -j=8
```

Chapter 6. Messages

6.1 Condition Value Format

Every routine in the subsystem returns a *condition value*: a longword that identifies both the outcome and its cause. The format is that used by OpenVMS and is declared by the UTILITY$ROUTINES package, which is required as an installed package rather than carried in this tree.

```
 31 30 29 28 27                16 15 14                 3 2      0
+--+--+--+--+--------------------+--+--------------------+-------+
|      CONTROL |      FAC_NO      |FS|       MSG_NO       |  SEV  |
+--+--+--+--+--------------------+--+--------------------+-------+
```

Field	Bits	Contents
`SEV`	0–2	Severity
`MSG_NO`	3–15	Message number within the facility
`FAC_NO`	16–31	Facility number
`CUST_DEF`	27	Set if the facility number was not assigned by the system vendor

The severity codes are:

Value	Severity	Success bit
0	Warning	clear
1	Success	set
2	Error	clear
3	Information	set
4	Severe error	clear

Bit 0 of the condition value is therefore the success indicator, and the outcome of any routine can be tested without regard to which condition it returned.

6.2 Message Format

Diagnostic messages have the following form:

```
28-JUL-2026 10:58:15 %BATCH-E-NOSCRIPT, script /nonexistent.sh not found - No such file or directory
```

The fields are the timestamp, the facility name, the severity mnemonic, the condition mnemonic, and the message text. When the `-v` option is in effect, a second line identifies the module and routine that detected the condition.

A condition value produced by the job controller is transmitted to the utility in the reply and is displayed by the utility as its own. The message reporting a nonexistent queue entry, for example, is printed by `batch` although the condition was detected by `batchd`.

6.3 Message Reference

Every message the subsystem issues is listed below, in the order the message numbers run. The text is that of the message record; a per cent sign in it marks where a value is inserted.

General

NORMAL, success

```
normal successful completion
```

```
*User action:* None.
```

INFO, informational

```
the controller runs on processors %s, and a job inherits that
```

```
*User action:* None. This is stated once when the controller starts.
```

BUGCHECK, severe

```
internal inconsistency: %s
```

```
*User action:* A defect of the subsystem. Report it with the accompanying text and the log of the controller.
```

General-purpose routines

BUFOVF, error

```
result of %d bytes does not fit in %zu
```

```
*User action:* Shorten the path or the value that did not fit. Where the paths are of the subsystem itself, re
```

INVTIME, error

```
unparsable time specification /AFTER=%s
```

```
*User action:* Correct the time specification. Appendix B states the forms accepted.
```

MKDIRERR, error

```
cannot create %s, mkdir()->%d, errno: %d (%s)
```

```
*User action:* Verify that the parent directory exists and that the controller may write in it.
```

IOERR, error

```
error transferring data, %s()->%d, errno: %d (%s)
```

```
*User action:* The connection to the other side failed. Retry; if it persists, examine the log of the control
```

ENDOFDATA, error

```
data ended earlier than expected, %s()->%d after %zu of %zu bytes
```

```
*User action:* The other side closed early. Retry. A client stopped with a signal produces this and is harmle
```

INVSTATE, warning

```
unknown job state
```

```
*User action:* A defect of the subsystem. Report it.
```

CWDERR, error

```
getcwd()->NULL, errno: %d (%s)
```

User action: The current directory has been removed or is unreadable. Change to a directory that exists.

SMALLBUF, error

```
supplied buffer is manifestly too small
```

User action: A defect of the subsystem. Report it.

Queue and jobs

JOBQUED, success

```
job %u (%s) queued on %s, state %s
```

User action: None. Note the entry number for DELETE and RELEASE.

JOBSTART, success

```
job %u (%s) started, pid=%d, log %s
```

User action: None.

JOBDONE, success

```
job %u (%s) completed successfully in %lld s
```

User action: None. The entry has left the queue.

JOBFAIL, error

```
job %u (%s) terminated with status %d, log %s
```

User action: Examine the log file named in the message.

JOBABORT, error

```
job %u (%s) terminated by signal %d, log %s
```

User action: The job was signalled. If it was not deleted deliberately, examine the log and the system jour

JOBORPHAN, warning

```
job %u was orphaned by a restart of the job controller
```

User action: The job was running when the controller stopped and its outcome is unknown. Submit it again, w

NOSLOT, error

```
the job table is full, capacity %d entries, while %s
```

User action: The queue is at its ceiling. Delete entries that are no longer wanted, or raise BATCH_K_MAXJOE

NOSUCHQUE, error

```
there is no queue named %s
```

User action: The queue is not one the controller was told of. SHOW /QUEUES lists them; the /QUEUE qualifier

NOTCONF, informational

```
not confirmed, nothing was done
```

User action: The question put by /CONFIRM was answered with something other than a word beginning with Y. N

PURGED, success

```
%u entries removed from the queue
```

```
*User action:* None. PURGE removed the entries of jobs that had finished.
```

QUEORPHAN, warning

```
queue %s was not configured; restored paused for its entries
```

```
*User action:* The journal held entries of a queue the controller was not told of. The queue was restored, an
```

QUEPAUSED, success

```
queue %s is paused
```

```
*User action:* None. The queue starts nothing until it is resumed.
```

SCRIPTCHG, error

```
entry %u: script %s was replaced since it was submitted
```

```
*User action:* The script is not the file that was submitted. It is pinned at submission by device, inode, si
```

SYNCFAIL, error

```
job %u ended with %s
```

```
*User action:* The job SYNC was waiting for did not succeed. Its log holds what the script said.
```

SYNCTMO, error

```
job %u has not ended within %u seconds
```

```
*User action:* The wait set by /TIMEOUT ran out. The job is still in the queue; SHOW will say what state it is
```

NOSUCHENT, error

```
no queue entry %u
```

```
*User action:* Verify the entry number with SHOW /ALL. An entry of a job that completed has left the queue.
```

NOPRIV, error

```
entry %u belongs to uid=%d, access denied
```

```
*User action:* The entry belongs to another user. Only its owner, or the privileged user, may act upon it.
```

NOTHELD, error

```
entry %u is not in the holding state
```

```
*User action:* RELEASE applies to a job in the Holding state. SHOW /ALL states the state of each entry.
```

ENTDELETED, success

```
entry %u deleted from the queue
```

```
*User action:* None.
```

ENTRELEASED, success

```
entry %u released from the holding state
```

```
*User action:* None.
```

ENTMODIFIED, success

```
entry %u modified: %s
```

```
*User action:* None. The list names the attributes that were altered; check it against what was typed.
```

NOTMODIFIABLE, error

```
entry %u is %s, the attribute cannot be altered now
```

User action: The state of the job forbids the change: an executing job admits /RETAIN, /KEEP and /RESTART o

NOTHINGSET, error

```
no attribute to set was given
```

User action: SET ENTRY was given no qualifier. Name the attribute to alter.

ABORTREQ, success

```
SIGTERM sent to job %u (pid=%d)
```

User action: None. The entry is disposed of when the job terminates.

JOBREQUEUED, success

```
job %u (%s) requeued for restart, attempt %u
```

User action: None. The job is being run again; BATCH_RESTART tells the job so.

Files and the queue journal

OPENIN, error

```
error opening %s for input, open()->%d, errno: %d (%s)
```

User action: Verify that the file exists and that the controller may read it.

OPENOUT, error

```
error opening %s for output, open()->%d, errno: %d (%s)
```

User action: Verify that the directory exists, that there is room on the file system, and that the controll

WRITEERR, error

```
error writing %s, errno: %d (%s)
```

User action: The file system is probably full. Make room and retry.

RENAMEERR, error

```
cannot rename %s to %s, rename()->%d, errno: %d (%s)
```

User action: The journal could not be put in place. Verify the room and the permissions on the subsystem ro

NOJOURNAL, informational

```
journal %s does not exist, the queue is empty
```

User action: None. The controller is starting with an empty queue, which is expected on a first start.

JRNCORRUPT, error

```
journal %s is corrupt: the header is missing
```

User action: The journal is unusable and the queue starts empty. Move the file aside and report it.

JRNVERSION, error

```
journal %s is incompatible: magic=%08x recsz=%u
```

User action: The journal was written by a different revision of the software. Remove it; the queue starts e

JRNTRUNC, warning

journal %s ends at record %u of %u

User action: The journal was cut short, probably by a full file system. The entries read are kept; the rest

JRNLOADED, informational

records loaded from the journal: %u

User action: None.

NOSCRIPT, error

script %s: %s, %s()->%d, errno: %d (%s)

User action: Verify the path, that the file exists, that it is a regular file, and that both you and the con

LOGSPEC, error

the job log specification cannot be constructed

User action: The log specification would exceed the room for it. Shorten the job name or the root directory

NODIR, error

directory %s is not accessible, realpath()->NULL, errno: %d (%s)

User action: Verify the directory named in /LOG_FILE.

LOGSWEPT, informational

%d job log%s removed from %s, older than %d day%s

User action: None. Set /LOG_RETAIN to alter the period, or to zero to stop the sweep.

Communication

NOCONNECT, error

job controller is not available, connect(%s)->%d, errno: %d (%s)

User action: The controller is not running, or the root directory differs. Start the service and verify BAT

SOCKERR, severe

socket(AF_UNIX)->%d, errno: %d (%s)

User action: The system refused a socket. Examine the limit on open files.

BINDERR, severe

bind(%s)->%d, errno: %d (%s)

User action: A stale socket file or a permission. Verify the subsystem root.

LISTENERR, severe

listen(%s)->%d, errno: %d (%s)

User action: Report it with the accompanying text.

ACCEPTERR, error

accept()->%d, errno: %d (%s)

User action: Usually transient. If it persists, examine the limit on open files.

PEERCRED, error

`getsockopt(SO_PEERCRED)->%d, errno: %d (%s)`

`*User action:* The identity of the client could not be established and the request is refused. Report it.`

BADMAGIC, error

`invalid frame signature: %08x`

`*User action:* Something other than the utility connected to the socket, or the two are of different revision`

BADLENGTH, error

`invalid %s payload length: %u`

`*User action:* The utility and the controller are of different revisions. Install them from one build.`

BADFUNC, error

`unknown function code %u`

`*User action:* The utility and the controller are of different revisions.`

NOREPLY, error

`no reply to the %s request`

`*User action:* The controller stopped while the request was in flight. Retry.`

SENDERR, error

`error transmitting %s`

`*User action:* The connection failed. Retry.`

RECVERR, error

`error receiving %s`

`*User action:* The connection failed. Retry.`

SOCKPATH, severe

`socket specification %s exceeds %zu bytes`

`*User action:* The root directory is too long for a socket path. Choose a shorter one.`

CONNCLOSED, warning

`client uid=%d closed the connection before sending a header`

`*User action:* None. A client that was stopped produces this.`

Command line

IVQUAL, error

`unrecognized or ambiguous qualifier %s`

`*User action:* Correct the qualifier. Chapter 5 lists those each command accepts.`

MISSVAL, error

`qualifier /%s requires a value`

`*User action:* Supply the value: the qualifier takes one, attached by an equals sign.`

IVQVAL, error

```
invalid value /RETAIN=%s
```

```
*User action:* Correct the value. The command entry states what the qualifier accepts.
```

IVPRIO, error

```
the value of /PRIORITY is outside 1..%d
```

```
*User action:* Give a priority between 1 and 255.
```

MAXPARM, warning

```
more than %d parameters, the excess were discarded
```

```
*User action:* The job was queued with the first eight parameters. Pass fewer, or pass them in a file.
```

NOPARAM, error

```
no script specification was given
```

```
*User action:* Supply the script specification.
```

EXTRAPARM, error

```
extraneous command parameter: %s
```

```
*User action:* A command takes one parameter at most. A mistyped qualifier is taken for a parameter, so check
```

IVCPU, error

```
unparsable processor list /CPU=%s
```

```
*User action:* Correct the processor list. The form is that of taskset(1): 0-3,5,8-11.
```

IVVERB, error

```
unrecognized command
```

```
*User action:* Use SUBMIT, SHOW, DELETE or RELEASE.
```

NOENTRY, error

```
no queue entry number was given
```

```
*User action:* Supply the entry number, positionally or with /ENTRY.
```

SUBREJECT, error

```
the job controller rejected the request, condition %08X
```

```
*User action:* The controller refused the request; its log states why in full.
```

Process creation and controller operation

FORKERR, error

```
job %u, fork()->%d, errno: %d (%s)
```

```
*User action:* The system could not create a process. Examine the limit on processes and the memory available
```

DAEMONERR, severe

```
daemon()->%d, errno: %d (%s)
```

```
*User action:* Report it with the accompanying text.
```

PIPEERR, severe

```
pipe()->%d, errno: %d (%s)
```

```
*User action:* Examine the limit on open files.
```

POLLERR, error

```
poll()->%d, errno: %d (%s)
```

```
*User action:* Report it with the accompanying text.
```

AFFERR, error

```
job %u: sched_setaffinity()->%d, errno: %d (%s)
```

```
*User action:* The processors named are not available to the controller. SHOW the mask it runs under in its l
```

UNKPROC, warning

```
an unknown process terminated, pid=%d
```

```
*User action:* None ordinarily. A process not belonging to any entry terminated.
```

CTLSTARTED, success

```
job controller started, root=%s, queue=%s, job limit=%d, socket=%s
```

```
*User action:* None.
```

CTLSTOPPING, success

```
job controller is stopping
```

```
*User action:* None.
```

IDENT, success

```
%s %s (%s), build %s
```

```
*User action:* None.
```

Chapter 7. Troubleshooting

Each section below states a symptom, what to look at, and what to do about it. Two places are worth knowing before any of them: the log of the controller, and the log of an individual job.

```
# journalctl -u batchd              # when the controller runs as a service
# cat /var/spool/batch/batchd.log   # when it was started by hand and detached
$ batch show /full                  # the state of every entry the queue holds
```

7.1 The Controller Cannot Be Reached

```
$ batch show
%BATCH-E-NOCONNECT, job controller is not available, connect(/var/spool/batch/batch.sock)-
>-1,
errno: 2 (No such file or directory)
```

The message names the socket the utility looked for. Take the causes in order.

The controller is not running.

```
# systemctl status batchd
# systemctl start batchd
```

It is running with a different root. The utility looks in `BATCH_ROOT`, or in the compiled-in default when that is unset; the controller uses whatever `/ROOT` it was given. Compare them:

```
$ echo "${BATCH_ROOT:-<unset, the built-in default is used>}"
# systemctl show batchd -p ExecStart | tr ' ' '\n' | grep ROOT
```

The socket exists but cannot be opened — `errno: 13 (Permission denied)`. The socket is created with mode 0666 and access is decided by the identity of the sender, so this means the *directory* denies passage. Verify that every component of the path to the root is traversable by the user.

7.2 The Controller Will Not Start

Read its first messages: it states the reason and stops.

```
%BATCH-F-BINDERR, bind(/var/spool/batch/batch.sock)->-1, errno: 98 (Address already in use)
```

Another controller is already running on that root. One controller owns one root; two would fight over the journal.

```
%BATCH-E-MKDIRERR, cannot create /var/spool/batch, mkdir()->-1, errno: 13 (Permission denied)
```

The account it runs under may not create the root. Either create it in advance with the right ownership, or run the controller as an account that may.

```
%BATCH-E-JRNVERSION, queue journal is incompatible with this revision
```

The journal was written by a different revision of the software. The queue starts empty; move the file aside or remove it. This is expected after an upgrade that altered the job descriptor.

7.3 A Job Does Not Start

`SHOW /ALL` states why: without `/ALL` a job that is merely waiting is not displayed at all.

```
$ batch show /all
 Entry  Job name             State      Submitted            Elapsed   Status
 -----  -------------------- ---------- -------------------- --------- ----------
     7  nightly              Pending    29-JUL-2026 10:01:05 00:00:00  -
```

State	What it means	What to do
`Pending`	Waiting for an execution slot	Others are running. `SHOW /ALL /EVERYONE` shows how many; raise `/JOB_LIMIT` if the limit is too tight.
`Holding`	Withheld by `/HOLD` or by `/AFTER`	`RELEASE` *entry* releases it. `SHOW /FULL` states the `/AFTER` time.
Not listed at all	The entry has left the queue	A job that succeeds leaves the queue. Submit with `/RETAIN=ALWAYS` if the entry is to remain.

A job with `/AFTER` in the past starts at once; one with `/AFTER` in the future waits, and the time displayed is the local one. A job that seems to wait longer than it was told to is worth checking against `TZ`: the controller and the enquirer may be reading different clocks.

7.4 A Job Fails

The entry stays in the queue with the outcome in the `Status` column, and the log file holds whatever the job wrote.

```
$ batch show
     7  nightly              Retained   29-JUL-2026 10:01:05 00:00:12  exit=127

$ batch show /full            # among the rest, the log file specification
$ cat /var/spool/batch/log/nightly_7.log
```

Status	Usual cause
`exit=126`	The job could not be started at all: the interpreter, the working directory, the log file or the processor list. See below.
`exit=127`	The interpreter did not find something the script called for. Usually `PATH`: a job is given `BATCH_*` and `P1` … `P8` and nothing else, so it does not inherit yours.
`exit=`*n*	The script itself returned that status. Its own affair.
`signal=15`	The job was deleted, or the system stopped it.
`signal=9`	Killed outright, commonly by the memory manager.
`exit=-1`	The controller stopped while the job ran and its outcome is unknown; the message is `%BATCH-W-JOBORPHAN`. Submit with `/RESTART` to have such a job run again.

`exit=126` is the one worth taking apart, since the job wrote nothing: the failure happened before the script began. In order of likelihood — the working directory of the submission no longer exists, the log file cannot be created, the processor list of `/CPU` names processors unavailable to the controller, or the identity of the submitter could not be assumed.

Remove the entry when done with it; the log goes with it unless `/KEEP` was given:

```
$ batch delete 7
```

7.5 The Queue Is Empty after a Restart

Expected in three cases, and a defect in none of them.

- A job that **succeeded** left the queue when it finished; that is the design. `/RETAIN=ALWAYS` keeps the entry.
- The journal was **refused**: `%BATCH-E-JRNVERSION` after an upgrade, or `%BATCH-E-JRNCORRUPT`. The controller says so at the start.
- The controller is running with a **different root** than the one that holds the journal.

A job that was *executing* is not lost but is marked `exit=-1` and stops there, unless it carried `/RESTART`, in which case it is queued again and told so through `BATCH_RESTART`.

7.6 The Log Directory Grows

Logs of jobs are removed with their queue entries. Two kinds survive that: those of jobs submitted with `/KEEP`, and those of jobs whose entries did not survive a hard stop. The controller sweeps them after `/LOG_RETAIN` days, seven by default, when it starts and once an hour thereafter.

```
%BATCH-I-LOGSWEPT, 5 job logs removed from /var/spool/batch/log, older than 7 days
```

Shorten the period if the directory still grows; nothing is removed while an entry in the queue owns it, however old it is. A log named by `/LOG_FILE` outside the log directory is never touched — that one is the business of whoever named it.

7.7 A Command Is Refused

```
$ batch submit /opt/jobs/nightly.sh /RE=ALWAYS
%CLI_RTNS-F: Ambiguous input 'RE' (matched to : 'RESTART', 'RETAIN')
```

Messages under `%CLI_RTNS-` come from the command line services and concern the form of the command; those under `%BATCH-` come from the subsystem and concern its meaning. An abbreviation must be unambiguous, and `/RESTART` and `/RETAIN` share their first two letters.

```
$ batch submit job.sh /NOSUCH=1
%BATCH-E-EXTRAPARM, extraneous command parameter: /NOSUCH=1
```

An argument beginning with a slash that names no qualifier is taken for a file specification, which is what allows `/tmp/job.sh` to be a path. A mistyped qualifier therefore appears as an extraneous parameter; check the spelling.

7.8 The Controller Is Slow to Answer

The queue is written to the journal in full whenever it changes, and the write is flushed to the medium. On a busy queue that is the dominant cost. If the controller lags under a burst of submissions, the queue is probably large: `SHOW /ALL /EVERYONE` says how large, and removing entries that are no longer wanted shortens every subsequent write.

A single client cannot hold the controller up: connections are served without blocking, and one that stops mid-request is dropped after thirty seconds.

7.9 Collecting Information for a Report

State the revision, the message in full, and what was being done:

```
$ batch                                    # the identification is in the first line
# journalctl -u batchd --since '1 hour ago' > batchd.txt
$ batch show /all /everyone /full > queue.txt
```

A message of severity `F` with the mnemonic `BUGCHECK` is a defect of the subsystem by definition and is always worth reporting.

Appendix A. Files

The subsystem root directory is established when the software is built and may be overridden by the `-r` option of the job controller and by the `BATCH_ROOT` environment variable for the utility. The default is `/var/spool/batch`.

File	Contents
root`/batch.sock`	The socket on which the job controller accepts requests
root`/journal.dat`	The queue journal
root`/log/`	Log files of jobs for which no `/LOG_FILE` was specified

The journal is written to a temporary file and renamed over the previous copy, so it is never observed in a partially written state. The journal contains a binary image of each queue entry; if the layout of the entry changes between versions, the journal of the earlier version is rejected and the queue starts empty.

The socket is created with mode 0666. Access to jobs is not controlled by file permissions but by the sender identification obtained from the kernel and verified for every request.

Environment Variables

Variable	Read by	Purpose
BATCH_ROOT	batch	Subsystem root directory
BATCH_ENTRY	Job	Entry number of the job
BATCH_NAME	Job	Job name
BATCH_QUEUE	Job	Queue name
BATCH_RESTART	Job	TRUE when the job is run again after a restart of the controller
BATCH_RESTARTS	Job	Number of times the job has been restarted
P1 … P8	Job	Job parameters

Appendix B. Time Specification Formats

The `/AFTER` qualifier accepts three forms of time specification.

B.1 Delta Time

A delta time begins with a plus sign and denotes an interval measured from the moment the command is entered:

```
+[dddd-]hh[:mm[:ss]]
```

Omitted fields are taken as zero.

Specification	Interval
`+3-`	Three days
`+3`	Three hours
`+:30`	Thirty minutes
`+::45`	Forty-five seconds
`+1-12:00`	One day and twelve hours
`+0-00:05:30`	Five minutes and thirty seconds

Note

The day field is told from the hour field by the hyphen that follows it, and by nothing else, so `+7` is seven hours and `+7-` is seven days. Since a mistake here is not apparent until the span has passed, the submission names the time the job is to start.

B.2 Absolute Time

An absolute time denotes a specific moment, read by the clock of whoever enters it:

```
dd-MMM-yyyy[:hh:mm:ss]
```

The month is a three-letter abbreviation in any case. Omitted time fields are taken as zero, so a date alone denotes midnight at the start of that day.

Specification	Meaning
`29-JUL-2026`	Midnight beginning 29 July 2026
`29-JUL-2026:03:00:00`	03:00 on 29 July 2026

B.3 Keywords

Keyword	Meaning
`NOW`	The current moment
`TODAY`	Midnight at the start of the current day
`TOMORROW`	Midnight at the start of the following day

Because `TODAY` denotes a moment that has already passed, a job submitted with `/AFTER=TODAY` is eligible for execution immediately.